

Objects

JS

Objects are the backbone of JavaScript: they hold most data, configs, and states. Mastering their manipulation is essential for writing clean, efficient, and bug-free code. Use this sheet to refresh key concepts quickly.

Core Actions:

Create:

```
const user = { name: 'Alice', age: 25 };
const empty = Object.create(null); // no prototype
```

Access:

```
user.name; // 'Alice' user
['age']; // 25
const key = 'name';
user[key]; // 'Alice'
```

Add / Modify / Delete:

```
user.city = 'Paris';
user.age = 26;
delete user.city;
```

Check existence:

```
'age' in user; // true
user.hasOwnProperty('age'); // true
```

Cloning & Merging:

Shallow copy:

```
const clone = { ...user };
const copy = Object.assign({}, user);
```

Deep copy:

```
const deep = structuredClone(user);
```

Merge:

```
const merged = { ...user, country: 'FR' };
Object.assign(user, { verified: true });
```

Inspect & Iterate:

Keys / Values / Entries:

```
Object.keys(user); // ['name', 'age'] Object.values(user); //
['Alice', 25] Object.entries(user); // [['name', 'Alice'], ['age', 25]]
```

Loop:

```
for (const key in user) { console.log(key, user[key]); }
Object.entries(user).forEach(([k, v]) => console.log(k, v));
for (const [k, v] of Object.entries(user)) { /* ... */ }
```

Transform:

```
const upper = Object.fromEntries( Object.entries(user).map(([k, v]) =>
[k.toUpperCase(), v]) );
```

Destructuring :

Extract values:

```
const { name, age } = user;
```

Nested:

```
const company = { info: { name: 'TechCorp', city: 'Paris' } };
const { info: { name, city } } = company;
```

Spread & Rest :

Spread (copy/merge):

```
const newUser = { ...user, age: 28 };
```

Rest (exclude props):

```
const { age, ...others } = user;
```

Utilities & Helpers:

Convert to JSON / Parse JSON:

```
const json = JSON.stringify(user);
const parsed = JSON.parse(json);
```

Get prototype:

```
Object.getPrototypeOf(user);
```

Define properties:

```
Object.defineProperty(user, 'id', { value: 1, writable: false });
```

Check equality (reference-based):

```
const a = { x: 1 };
const b = { x: 1 };
a === b; // false (different refs)
```

Compare deeply:

```
JSON.stringify(a) === JSON.stringify(b); // true (simple deep check)
```

Common Patterns:

Default values:

```
const config = { retries: 3, timeout: 1000 };
const opts = { ...config, ...userOptions };
```

Optional chaining:

```
const city = user.address?.city ?? 'Unknown';
```

Dynamic property names:

```
const prop = 'role';
const obj = { [prop]: 'admin' };
```

Pro Tips :

- ✓ Use `structuredClone()` for safe deep copies.
- ✓ Avoid mutating objects passed as function params.
- ✓ For config merges, prefer `{ ...defaults, ...overrides }`.
- ✓ Combine with arrays via `Object.entries()` + `map()` magic.

Brought to you by

 **certificates.dev**

Learn more at certificates.dev/javascript

Creators of the

 **JavaScript Certification**

Objects

JS

Objects are the backbone of JavaScript: they hold most data, configs, and states. Mastering their manipulation is essential for writing clean, efficient, and bug-free code. Use this sheet to refresh key concepts quickly.

Core Actions:

Create:

```
const user = { name: 'Alice', age: 25 };
const empty = Object.create(null); // no prototype
```

Access:

```
user.name; // 'Alice' user
['age']; // 25
const key = 'name';
user[key]; // 'Alice'
```

Add / Modify / Delete:

```
user.city = 'Paris';
user.age = 26;
delete user.city;
```

Check existence:

```
'age' in user; // true
user.hasOwnProperty('age'); // true
```

Cloning & Merging:

Shallow copy:

```
const clone = { ...user };
const copy = Object.assign({}, user);
```

Deep copy:

```
const deep = structuredClone(user);
```

Merge:

```
const merged = { ...user, country: 'FR' };
Object.assign(user, { verified: true });
```

Inspect & Iterate:

Keys / Values / Entries:

```
Object.keys(user); // ['name', 'age']
Object.values(user); // ['Alice', 25]
Object.entries(user); // [['name', 'Alice'], ['age', 25]]
```

Loop:

```
for (const key in user) { console.log(key, user[key]); }
Object.entries(user).forEach(([k, v]) => console.log(k, v));
for (const [k, v] of Object.entries(user)) { /* ... */ }
```

Transform:

```
const upper = Object.fromEntries(
  Object.entries(user).map(([k, v]) => [k.toUpperCase(), v])
);
```

Destructuring :

Extract values:

```
const { name, age } = user;
```

Nested:

```
const company = { info: { name: 'TechCorp', city: 'Paris' } };
const { info: { name, city } } = company;
```

Spread & Rest :

Spread (copy/merge):

```
const newUser = { ...user, age: 28 };
```

Rest (exclude props):

```
const { age, ...others } = user;
```

Utilities & Helpers:

Convert to JSON / Parse JSON:

```
const json = JSON.stringify(user);
const parsed = JSON.parse(json);
```

Get prototype:

```
Object.getPrototypeOf(user);
```

Define properties:

```
Object.defineProperty(user, 'id', { value: 1, writable: false });
```

Check equality (reference-based):

```
const a = { x: 1 };
const b = { x: 1 };
a === b; // false (different refs)
```

Compare deeply:

```
JSON.stringify(a) === JSON.stringify(b); // true (simple deep check)
```

Common Patterns:

Default values:

```
const config = { retries: 3, timeout: 1000 };
const opts = { ...config, ...userOptions };
```

Optional chaining:

```
const city = user.address?.city ?? 'Unknown';
```

Dynamic property names:

```
const prop = 'role';
const obj = { [prop]: 'admin' };
```

Pro Tips :

- ✓ Use `structuredClone()` for safe deep copies.
- ✓ Avoid mutating objects passed as function params.
- ✓ For config merges, prefer `{ ...defaults, ...overrides }`.
- ✓ Combine with arrays via `Object.entries()` + `map()` magic.

Brought to you by



Learn more at certificates.dev/javascript

Creators of the

